ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 22

# Calidad y buenas prácticas

- ¿Qué es código de calidad?
- Nombres que se explican solos
- Funciones cortas: DRY, KISS y YAGNI
- Estilo común: PSR-12
- Manejo global de errores
- Inyección de dependencias, sin complicaciones
- Los principios SOLID
- Para saber más: herramientas que revisan por vosotros
- Para saber más: patrones de diseño



**Ramón Abramo**

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

# 🧭 Contenido de la clase

Hasta ahora nos hemos preocupado de que el código **funcione**. Hoy damos un paso más: que funcione **y que se pueda mantener**. Porque el código se escribe una vez, pero se lee decenas de veces: cuando hay que corregir un fallo, añadir algo o explicárselo a un compañero. Y muchas veces ese compañero sois vosotros mismos, seis meses después, sin acordaros de nada. 😅

Esta clase no trae instrucciones nuevas de PHP. Trae **costumbres**: las que distinguen al programador que hace que algo funcione del que hace algo que otros pueden mantener. Y prepara el terreno para lo que viene: construir nuestro propio *framework* MVC.

**1**

📘 ¿Qué es código de calidad?



**2**

🔍 Nombres que se explican solos



**3**

👍 Funciones cortas: DRY, KISS y YAGNI



**4**

👌 Estilo común: PSR-12



**5**

🐞 Manejo global de errores



**6**

🚀 Inyección de dependencias, sin complicaciones



**7**

⚠️ Los principios SOLID



**8**

😮 Para saber más: herramientas que revisan por vosotros



**9**

🔍 Para saber más: patrones de diseño



**10**

🚀 Ejercicios



**11**

👋 Lo que tenéis que llevaros de hoy





CLASE 22 · PUNTO 1

## 📘 ¿Qué es código de calidad?

Un código de calidad no es el más corto ni el más ingenioso. Es el que cumple tres cosas:

| Se puede… | Pregunta para comprobarlo |
|---|---|
| **Leer** | ¿Un compañero entiende qué hace sin que se lo expliquéis? |
| **Cambiar** sin miedo | ¿Podéis añadir algo sin romper otra cosa que no tiene nada que ver? |
| **Probar** | ¿Podéis comprobar que funciona sin montar toda la aplicación? |

Todo lo de hoy sirve para eso. Y ya habéis aplicado muchas de estas ideas sin ponerles nombre: las funciones de la clase 10, el orden funciones → inicialización → entradas → procesamiento → salidas, los repositorios de la clase 21…



CLASE 22 · PUNTO 2

## 🔍 Nombres que se explican solos

¿Qué hace esta función?

01-nombres.php — php-paso-a-paso/clase22 — Visual Studio Code

01-nombres.php

```php
<?php
function calc(array $d, float $x): float
{
    $t = 0;
    foreach ($d as $i) {
        if ($i["p"] > 0) {
            $t += $i["p"] * $i["c"];
        }
    }
    return $t * (1 + $x);
}
```

Hay que leerla entera y adivinar. Ahora, **el mismo código** con buenos nombres:

01-nombres.php — php-paso-a-paso/clase22 — Visual Studio Code

01-nombres.php

```php
<?php
function calcularTotalConIva(array $lineasDelPedido, float $tipoIva): float
{
    $subtotal = 0;
    foreach ($lineasDelPedido as $linea) {
        if ($linea["precio"] > 0) {
            $subtotal += $linea["precio"] * $linea["cantidad"];
        }
    }
    return $subtotal * (1 + $tipoIva);
}
```

📄 Ejemplo: `01-nombres.php`

| Regla | ❌ | ✅ |
|---|---|---|
| Nombres completos, sin abreviaturas raras | `$d`, `$t`, `calc()` | `$lineas`, `$subtotal`, `calcularTotal()` |
| Variables y propiedades: **sustantivos** | `$calcular` | `$total` |
| Funciones y métodos: **verbos** | `function total()` en algo que guarda | `guardar()`, `calcularTotal()` |
| Las que devuelven `bool`: **preguntas** | `function aprobado()` | `haAprobado()`, `estaCompleto()`, `tieneStock()` |
| Números mágicos con nombre | `if ($nota >= 5)` repartido por el código | `const NOTA_PARA_APROBAR = 5;` |



🎯 Si para entender una línea necesitáis un comentario, probad primero a **cambiar los nombres**. Muchas veces el comentario sobra.



CLASE 22 · PUNTO 3

## 👍 Funciones cortas: DRY, KISS y YAGNI

Una función que valida, calcula, aplica descuentos y formatea es difícil de leer y de probar. La solución: **una función, una tarea**.

02-funciones-cortas.php — php-paso-a-paso/clase22 — Visual Studio Code

02-funciones-cortas.php

```php
<?php
function resumirPedido(array $pedido): string
{
    $errores = erroresDelPedido($pedido);
    if (count($errores) > 0) {
        return "Error: " . implode(", ", $errores);
    }
    return "Pedido de {$pedido['cliente']}: " .
euros(aplicarDescuento(subtotal($pedido["lineas"])));
}
```

Cada pieza (`erroresDelPedido()`, `subtotal()`, `aplicarDescuento()`, `euros()`) hace una sola cosa, cabe en la pantalla y se puede reutilizar.

📄 Ejemplo: `02-funciones-cortas.php`

Tres siglas que oiréis mucho:

| Sigla | En inglés | Significa |
|---|---|---|
| **DRY** | *Don't Repeat Yourself* | No repitáis código: si lo copiáis por tercera vez, falta una función o una clase |
| **KISS** | *Keep It Simple, Stupid* | La solución más sencilla que funcione suele ser la mejor |
| **YAGNI** | *You Aren't Gonna Need It* | No programéis cosas «por si acaso». Cuando hagan falta, se añaden |



💡 YAGNI es el antídoto contra un vicio muy común: llenar el código de opciones que nadie ha pedido. Cada línea que escribís es una línea que hay que mantener.



CLASE 22 · PUNTO 4

## 👌 Estilo común: PSR-12

¿Las llaves van en la misma línea o en la siguiente? ¿Cuatro espacios o un tabulador? Si cada uno lo hace a su manera, el código de un equipo parece escrito por diez personas distintas... porque lo está. Para evitarlo, la comunidad PHP tiene un estándar de estilo: **PSR-12** [1].

03-psr12-antes.php — php-paso-a-paso/clase22 — Visual Studio Code

03-psr12-antes.php

```php
<?php
// ❌ Funciona, pero no sigue PSR-12
class producto{
  public $Nombre;
    function __construct($n,$p){$this->Nombre=$n;$this->precio=$p;}
  public function PrecioConIva()
  {
    if($this->precio>0){ return $this->precio*1.21; }
    else return 0;
  }
}
```

| Regla de PSR-12 | Ejemplo |
|---|---|
| Sangrado de **4 espacios** (nunca tabuladores) | |
| Clases en `PascalCase`; métodos y propiedades en `camelCase` | `class Producto`, `precioConIva()` |
| La llave `{` de clases y métodos, en la **línea siguiente** | `class Producto` ↵ `{` |
| La llave de `if`, `for`, `foreach`… en la **misma línea**, con espacio | `if ($precio > 0) {` |
| Un espacio alrededor de los operadores | `$precio * 1.21`, no `$precio*1.21` |
| Siempre llaves, aunque el bloque tenga una línea | Nada de `else return 0;` |
| Visibilidad (`public`, `private`) **siempre** indicada | `public function`, no solo `function` |
| Una línea en blanco después de `<?php` y de `declare` en archivos de clases | |

📄 Ejemplos: `03-psr12-antes.php` y `04-psr12-despues.php`



🎯 No hace falta memorizar las reglas: el editor puede formatear por vosotros. En VS Code, con la extensión de PHP que instalamos en la clase 3, **Mayús + Alt + F** formatea el archivo. Y **PHP-CS-Fixer** (punto 8) revisa todo un proyecto de golpe.



CLASE 22 · PUNTO 5

# 🐞 Manejo global de errores

En la clase 14 aprendisteis a capturar excepciones con `try` y `catch`. Pero ¿qué pasa con las que **nadie** captura? Que el visitante ve una página en blanco o, peor, el mensaje de error con rutas de vuestro servidor.

Con `set_exception_handler()` podéis registrar una función que recoja **cualquier** excepción que se escape, esté donde esté [2]:

05-manejador-global.php — php-paso-a-paso/clase22 — Visual Studio Code

🐘 05-manejador-global.php

```php
<?php
function manejarExcepcion(Throwable $e): void
{
    // Para nosotros: todos los detalles en el registro
    error_log(sprintf("[%s] %s en %s:%d", get_class($e), $e->getMessage(), $e->getFile(),
$e->getLine()));

    // Para el visitante: un mensaje amable
    if (!headers_sent()) {
        http_response_code(500);
    }
    echo "<h1>Vaya, algo ha fallado 😕</h1>";
}

set_exception_handler("manejarExcepcion");
```

Y con `set_error_handler()` convertís también los avisos (*warnings*) en excepciones, para que tampoco se escapen:

05-manejador-global.php — php-paso-a-paso/clase22 — Visual Studio Code

🐘 05-manejador-global.php

```php
<?php
function avisoAExcepcion(int $nivel, string $mensaje, string $archivo, int $linea): bool
{
    throw new ErrorException($mensaje, 0, $nivel, $archivo, $linea);
}

set_error_handler("avisoAExcepcion");
```

📄 Ejemplo: `05-manejador-global.php` . Ejecutadlo y abrid el `errores.log` que aparece en la carpeta.



⚠️ Fijaos en que el ejemplo **calcula todo antes de mostrar nada**: nuestro orden de siempre. Si el fallo llega a mitad de la salida, el visitante vería media página y el código 500 ya no se podría enviar. En el MVC, este manejador será lo primero que se registre.



CLASE 22 · PUNTO 6

## 🚀 Inyección de dependencias, sin complicaciones

El nombre asusta, pero la idea ya la usasteis en la clase 21, cuando el repositorio recibía la conexión en su constructor. Comparad:

06-inyeccion-dependencias.php — php-paso-a-paso/clase22 — Visual Studio Code

06-inyeccion-dependencias.php

```php
<?php
// ❌ La clase fabrica su dependencia con new: está «soldada» al email
class MatriculaRigida
{
    public function confirmar(string $email): string
    {
        $notificador = new NotificadorEmail();
        return $notificador->enviar($email, "¡Matrícula confirmada!");
    }
}

// ✅ La clase dice qué necesita y otro se lo da: la dependencia se INYECTA
class Matricula
{
    public function __construct(private Notificador $notificador)
    {
    }

    public function confirmar(string $email): string
    {
        return $this->notificador->enviar($email, "¡Matrícula confirmada!");
    }
}

$deVerdad = new Matricula(new NotificadorEmail());
$deMentira = new Matricula(new NotificadorPrueba());   // para pruebas: no envía nada
```

La analogía: un taladro con la broca **soldada** frente a uno con **portabrocas**. El segundo os vale para madera, metal o pared; basta con cambiar la broca.

📄 Ejemplo: `06-inyeccion-dependencias.php`



💡 En el MVC construiremos un **contenedor de dependencias**: una pieza que se encarga de crear los objetos y pasarles lo que necesitan. Laravel tiene uno muy potente, y ahora sabréis qué hace por dentro.



CLASE 22 · PUNTO 7

## ⚠️ Los principios SOLID

**SOLID** son cinco principios de diseño orientado a objetos, uno por cada letra [3]. No son leyes: son consejos que, aplicados con sentido común, hacen que el código aguante bien los cambios.

| Letra | Principio | En una frase |
|---|---|---|
| **S** | Responsabilidad única | Una clase debe tener **un solo motivo para cambiar** |
| **O** | Abierto/cerrado | Abierto a **añadir** comportamiento, cerrado a **modificar** lo que ya funciona |
| **L** | Sustitución de Liskov | Una clase hija debe poder usarse donde se usa la padre sin sorpresas |
| **I** | Segregación de interfaces | Mejor varias interfaces pequeñas que una enorme que obligue a escribir métodos que no se usan |
| **D** | Inversión de dependencias | Depended de **interfaces**, no de clases concretas (lo del punto 6) |

### S · Responsabilidad única

Una `Factura` que calcula el total, genera el HTML **y** se guarda en un fichero tiene tres motivos para cambiar: las reglas de cálculo, el diseño y el almacenamiento. Mejor tres clases:

07-solid-responsabilidad-unica.php — php-paso-a-paso/clase22 — Visual Studio Code

07-solid-responsabilidad-unica.php

```php
<?php
class Factura { public function total(): float { /* ... */ } }
class FacturaHtml { public function mostrar(Factura $factura): string { /* ... */ } }
class FacturaRepositorio { public function guardar(Factura $factura, int $numero): void { /*
... */ } }
```

Os sonará: es exactamente la separación entre **entidad** y **repositorio** de la clase 21.

📄 Ejemplo: `07-solid-responsabilidad-unica.php`

## O · Abierto/cerrado

Una función con un `if/elseif` por cada tipo de descuento hay que **abrirla y tocarla** con cada descuento nuevo, con el riesgo de estropear los que ya funcionaban. Con una interfaz, cada descuento es una clase y la función no se toca nunca:

08-solid-abierto-cerrado.php — php-paso-a-paso/clase22 — Visual Studio Code

08-solid-abierto-cerrado.php

```php
<?php
interface Descuento
{
    public function aplicar(float $precio): float;
}

function precioFinal(float $precio, Descuento ...$descuentos): float
{
    foreach ($descuentos as $descuento) {
        $precio = $descuento->aplicar($precio);
    }
    return round($precio, 2);
}

echo precioFinal(250, new DescuentoEstudiante(), new DescuentoAntiguoAlumno());   // 180
```

¿Os suena? Es el **polimorfismo** de la clase 18. Los `...` delante de `$descuentos` significan «tantos descuentos como queráis»: es una función **variádica**.

📄 Ejemplo: `08-solid-abierto-cerrado.php`



🎯 Mi consejo: no intentéis aplicar los cinco principios a todo desde el primer día (recordad KISS y YAGNI). Cuando un cambio pequeño os obligue a tocar muchos sitios, ese es el momento de preguntaros cuál de ellos estáis incumpliendo.



CLASE 22 · PUNTO 8

## 😮 Para saber más: herramientas que revisan por vosotros

Los profesionales no revisan todo a mano: usan herramientas que se instalan con Composer **solo para desarrollo** (`composer require --dev`). En `ejemplos/herramientas/` tenéis un proyecto pequeño preparado. Entrad en su carpeta desde la terminal de Laragon y ejecutad `composer install`.

| Herramienta | Qué hace | Comando |
|---|---|---|
| **PHPStan** [4] | Lee el código **sin ejecutarlo** y encuentra errores: métodos que no existen, tipos que no encajan, variables que pueden ser `null` | `composer analizar` |
| **PHPUnit** [5] | Ejecuta **pruebas automáticas**: pequeños programas que comprueban que vuestro código hace lo que debe | `composer probar` |
| **PHP-CS-Fixer** [6] | Revisa (y corrige) el estilo PSR-12 de todo el proyecto | `composer estilo` |

Los nombres `analizar`, `probar` y `estilo` son **scripts** del `composer.json`, los atajos de la clase 20.

## Una prueba automática

Precios.php — php-paso-a-paso/clase22 — Visual Studio Code

Precios.php

```php
<?php
class PreciosTest extends TestCase
{
    public function testAnadeElIvaGeneral(): void
    {
        $this->assertSame(121.0, Precios::conIva(100));
    }

    public function testRechazaUnDescuentoMayorQueCien(): void
    {
        $this->expectException(InvalidArgumentException::class);

        Precios::conDescuento(50, 150);
    }
}
```

Cada método que empieza por `test` es una prueba: **prepara** los datos, **ejecuta** el código y **comprueba** el resultado con un `assert…`. Si mañana alguien rompe `conIva()` sin querer, `composer probar` lo dirá en un segundo.

📄 Archivos: `src/Precios.php` y `tests/PreciosTest.php`



CLASE 22 · PUNTO 9

## 🔍 Para saber más: patrones de diseño

Un **patrón de diseño** es una solución con nombre propio a un problema que aparece una y otra vez. Conocer los nombres sirve, sobre todo, para entenderos con otros programadores. Algunos ya los habéis usado sin saberlo:

| Patrón | Idea | Dónde lo habéis visto |
|---|---|---|
| **Repository** | Una clase que guarda y recupera objetos, y esconde el SQL | `CursoRepositorio`, clase 21 |
| **Singleton** | Una sola instancia compartida | `Conexion::obtener()`, clase 21 |
| **Factory** (fábrica) | Un método que fabrica objetos | `Curso::desdeFila()`, `Dinero::euros()` |
| **Strategy** (estrategia) | Varias formas de hacer algo, intercambiables tras una interfaz | Los descuentos de esta clase, los notificadores |
| **Observer** (observador) | Varios objetos «se suscriben» para enterarse cuando pasa algo | Los eventos de Laravel (más adelante) |



CLASE 22 · PUNTO 10

## 🚀 Ejercicios

- ☐ **01 · Refactorizar.** El archivo `ej01-antes.php` funciona, pero no hay quien lo lea. Dejadlo con buenos nombres, funciones con tipos, una constante para la nota de aprobado y estilo PSR-12, sin cambiar el resultado. → `ej01-despues.php`
- ☐ **02 · Envíos.** Calculad el coste de enviar un paquete según el peso con estándar, urgente y recogida en tienda, aplicando el principio **abierto/cerrado**: una interfaz `MetodoEnvio` y una clase por método. → `ej02-envios.php`
- ☐ **03 · Informe de ventas.** Una clase `Informe` que diga el producto más vendido y el total, y que funcione igual con ventas en memoria o leídas de un CSV. Pista: **inversión de dependencias**, con una interfaz `FuenteDeVentas`. → `ej03-informe.php`
- ☐ **04 · Una prueba más.** En el proyecto de herramientas, añadid una prueba que compruebe que un descuento del 0 % deja el precio igual y otra para un descuento negativo, y ejecutad `composer probar`. → `ej04-PreciosTest.php`



🏆 **Reto:** pasad PHPStan (`composer analizar`) por alguna de vuestras clases de las clases 17 a 21, copiándola en `src/`. ¿Encuentra algo? Leed con calma cada aviso: casi siempre tiene razón.



CLASE 22 · PUNTO 11

## 👋 Lo que tenéis que llevaros de hoy

| Idea | En una frase |
|---|---|
| Calidad | Código que se lee, se cambia sin miedo y se puede probar |
| Nombres | Sustantivos para datos, verbos para acciones y preguntas para los `bool` |
| Funciones | Cortas y con una sola tarea |
| DRY, KISS y YAGNI | No repetir, no complicar y no programar «por si acaso» |
| PSR-12 | El estilo común de PHP; que lo aplique el editor |
| Errores globales | `set_exception_handler()` y `set_error_handler()`: nunca una página en blanco |
| Inyección de dependencias | Las clases reciben lo que necesitan; no lo fabrican con `new` |
| SOLID | Una responsabilidad, abierto a añadir, depender de interfaces… |
| Herramientas ⭐ | PHPStan analiza, PHPUnit prueba y PHP-CS-Fixer ordena |

# 📚 Fuentes y documentación oficial

1. PHP-FIG: PSR-12 Extended Coding Style
2. Manual de PHP: set_exception_handler
3. Robert C. Martin: The Principles of OOD
4. PHPStan: Getting Started
5. PHPUnit: documentación
6. PHP-CS-Fixer